

# Technical Task – Backend Engineer

## Sensor Ingestion, Stateful Rule Engine & Alerting

---

### Table of Contents

|                                      |   |
|--------------------------------------|---|
| 1 Executive Summary .....            | 1 |
| 2 Problem Statement.....             | 1 |
| 3 Out of Scope.....                  | 2 |
| 4 Input Data.....                    | 2 |
| 5 Deliverables .....                 | 2 |
| 6 Rule Engine Requirements .....     | 3 |
| 7 Alerting Requirements .....        | 4 |
| 8 Aggregation API.....               | 5 |
| 9 Processing Report .....            | 5 |
| 10 Architecture Expectations.....    | 5 |
| 11 Evaluation Criteria.....          | 5 |
| 12 Submission & Given Material ..... | 5 |

## 1 Executive Summary

This document is the Backend Engineer technical task issued by danatadbir (i4Twins). The scope is designed to fit within two working days and focuses on three areas: a stateful rule operator, SustainedAbove, which requires explicit state handling and tolerance for out-of-order data; an alerting layer with cooldown-based deduplication instead of classification-only output for sustained rules; and idempotent processing, where re-running the same input file must not create duplicate data or alerts. Rules are not managed through an API – they are supplied as seed data in rules.json. Candidates choose the storage engine, architectural style, and language (C#/.NET 8 or 10 preferred).

## 2 Problem Statement

Build a small backend service that ingests numeric sensor readings from a file, cleans duplicate and invalid records, evaluates valid readings against a set of user-defined rules (including one time-based, stateful operator), classifies readings as acceptable or unacceptable, produces alerts for sustained-rule violations, and exposes time-based aggregation of the acceptable data via an API. The task assesses messy-data handling,

stateful business-rule evaluation, domain modelling, clean and extensible architecture, data processing, and overall engineering quality.

### 3 Out of Scope

Keep the solution focused. The following are explicitly out of scope: message queues, brokers, or Kubernetes (reading from the provided file is sufficient); authentication; any user interface; production deployment; and rule-management CRUD endpoints – rules are provided as seed data. If you believe the task genuinely needs more than two working days, say so early and explain why.

### 4 Input Data

The input is `readings.jsonl`, one JSON reading per line – roughly 2,150 readings across several devices and metrics, supplied with the task:

| Field    | Type                  | Example                            |
|----------|-----------------------|------------------------------------|
| deviceld | string                | PUMP-01                            |
| metric   | string                | temperature / pressure / vibration |
| ts       | string (ISO-8601 UTC) | 2025-06-01T08:33:00Z               |
| value    | number                | 67.21                              |
| seq      | integer               | 1199                               |

Handling messy data is **the core of the task**:

- **Duplicates:** a reading is a duplicate of another when the **(deviceld, metric, ts, seq)** quadruple is equal. Design and justify your duplicate policy (e.g., first-wins vs last-wins).
- **Ordering:** the file is **not time-sorted**. Out-of-order readings are expected, and your stateful rule evaluation must behave correctly and predictably in their presence – sort first, use event-time windowing, or another approach; your choice, but it must be deliberate and documented, including how late arrivals are treated.
- **Invalid records:** malformed lines or semantically invalid readings are rejected and counted without crashing the service.
- **Separation of concerns:** a rule violation is **not** malformed data. Invalid and duplicate readings are rejected before evaluation and must **not** be counted as unacceptable for that reason.

### 5 Deliverables

1. **Ingestion:** read `readings.jsonl` (one JSON reading per line).

2. **Storage:** persist processed readings, rule results, and alerts — storage of your choice (in-memory, SQLite, etc.), justified in the README.
3. **Rules as seed data:** rules are defined in a `rules.json` file loaded at startup; design the rule model so that adding, enabling, or disabling a rule is a data change, not a code change.
4. **Rule evaluation:** every valid, non-duplicate reading is classified as acceptable or unacceptable, with violated rule(s) identified and a meaningful reason.
5. **Alerting:** sustained-rule violations produce alerts with start/end timestamps and cooldown-based deduplication (see *Alerting Requirements*).
6. **Acceptable/unacceptable outputs:** maintain separate lists; unacceptable readings are excluded from aggregation.
7. **Aggregation API:** a GET endpoint returning per-time-bucket aggregates of acceptable data per device + metric.
8. **Idempotent re-run:** processing the same input file twice must not create duplicate readings, rule results, or alerts. State the keying strategy you use and demonstrate it in a test.
9. **Processing report:** at the end of processing, print the counts listed under *Processing Report*, plus meaningful logging of significant events.
10. **Tests:** at minimum, unit tests covering deduplication, rule evaluation (including `SustainedAbove` with out-of-order data), alert cooldown, idempotent re-run, and aggregation.
11. **README:** how to run and test; key architectural decisions and trade-offs; rule model and evaluation policy; state and ordering strategy for `SustainedAbove`; alert cooldown policy; aggregation behavior; **disclosure of any AI tool usage**.

## 6 Rule Engine Requirements

Rules apply to a metric and optionally to a device; a rule without a `deviceId` applies to all devices carrying that metric. A rule has an identifier, name, enabled/disabled flag, metric, optional `deviceId`, operator, and the operator's required value(s).

**Operators (minimum set):** `GreaterThan`, `GreaterThanOrEqual`, `LessThan`, `LessThanOrEqual`, `Equal`, **Between** — plus the stateful operator below.

**SustainedAbove (required, stateful):**

```
{
  "name": "Overheating",
  "deviceId": "PUMP-01",
  "metric": "temperature",
  "operator": "SustainedAbove",
```

```

"threshold": 80,
"durationSeconds": 30,
"enabled": true
}

```

A SustainedAbove rule is violated when the metric stays above threshold for at least durationSeconds of *event time*. This requires maintaining state per (**deviceId**, **metric**) and forces explicit, documented decisions about:

- how out-of-order readings are handled (the file is not time-sorted);
- how the duration window is defined and measured (event time, not processing order);
- how the violation window starts and ends (start when the value first crosses the threshold, end when it drops back to or below it);
- whether the evaluation is streaming (incremental state) or batch (sort then scan) — either is acceptable, but the trade-offs must be explained in the README.

**Classification:** a reading is acceptable when it satisfies all applicable enabled rules; it is unacceptable when at least one rule is violated, with the rule identified and an explanation. If no enabled rule applies, document your policy (recommended: acceptable). Invalid and duplicate readings never reach evaluation and are never counted as unacceptable.

**Extensibility:** the engine must be open to adding new operators without modifying existing evaluation code; demonstrate this by design and explain in the README how a new operator would be added.

## 7 Alerting Requirements

For SustainedAbove (and any future stateful operator), the output is not a per-reading classification but an **alert record** containing at least: rule identifier, deviceId, metric, alert **startTs** (when the sustained condition began), **endTs** (when it ended, or the end of the observed data), and the peak value if you choose to include it.

**Cooldown (required):** after an alert is raised for a given (**rule**, **deviceId**, **metric**), no new alert may be produced for the same combination until the cooldown period has elapsed — use **5 minutes** unless you justify a different value in the README. This is deduplication at the *domain* level: one sustained episode maps to one alert, not one alert per offending reading. Episodes separated by more than the cooldown produce separate alerts.

Alerts must be stored alongside the other outputs and must respect the idempotency requirement: re-running the same file must not produce a second copy of any alert.

## 8 Aggregation API

Provide a GET endpoint (route and parameter names are your choice; correctness matters) with inputs: deviceId, metric, from, to, and bucket size (e.g., seconds). The output contains, for each equally sized bucket within [from, to): the bucket start time, count, average, min, and max – computed over **acceptable readings only**. Whether empty buckets are omitted or reported with zero counts is your choice; document it briefly.

## 9 Processing Report

At the end of processing, print a report with at least: total lines read, parsed readings, stored readings, duplicates removed, invalid records rejected, rules loaded, rule evaluations performed, acceptable readings, unacceptable readings, rule violations, and alerts generated. The numbers must reflect the real input. Log significant events (rejections, sustained episodes, alerts raised) at appropriate levels.

## 10 Architecture Expectations

Domain logic must be independent of infrastructure (Clean Architecture, Layered, or Vertical Slice – your choice, explained in the README). Validation, deduplication, rule applicability, rule evaluation, sustained-state management, alerting, and classification belong in the domain/application layer – not in controllers or repositories. The design must be extensible for new aggregations, new input sources, and new operators, with clear separation of concerns and reasonable time and memory complexity; document your trade-offs.

## 11 Evaluation Criteria

Submissions are graded on: (1) robustness against messy data – dedup, invalid records, out-of-order ordering; (2) rule-engine correctness – applicability, all operators including the stateful SustainedAbove, classification, and violation information; (3) alerting correctness – episode detection, timestamps, and cooldown deduplication; (4) idempotency – safe re-processing of the same input; (5) aggregation correctness using only acceptable readings; (6) code cleanliness – naming, structure, error handling, readability; (7) test quality and coverage; (8) design and architecture – separation of concerns, domain independent of infrastructure, extensibility, SOLID where appropriate; (9) README clarity and your ability to explain the design in an online session.

## 12 Submission & Given Material

Submit a Git repository with an incremental, meaningful commit history – not a single final commit. AI tools are allowed but must be disclosed in the README. A short online walkthrough session is part of the process; the final deliverable is the repository address

plus the README. The task ships with `readings.jsonl` (~2,150 readings across several devices and metrics) and `rules.json` seed data.